

Final Term

12. Number system-> Binary to Decimal and Decimal to Binary

Binary -> 2 base (1101_2)

Decimal -> 10 base (13_{10})

Hexadecimal -> 16 base (62_c)

Octal -> 8 base (154_8)

Binary to Decimal convert:

Find the decimal value of 111001_2 :

binary number:	1	1	1	0	0	1
power of 2:	2^5	2^4	2^3	2^2	2^1	2^0

$$111001_2 = 1 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 = 57_{10}$$

Decimal to Binary convert:

Find the binary value of 13_{10} :

Convert 13_{10} to binary:

Division by 2	Quotient	Remainder	Bit #
$13/2$	6	1	0
$6/2$	3	0	1
$3/2$	1	1	2
$1/2$	0	1	3

So $13_{10} = 1101_2$

Convert 174_{10} to binary:

Division by 2	Quotient	Remainder	Bit #
$174/2$	87	0	0
$87/2$	43	1	1
$43/2$	21	1	2
$21/2$	10	1	3
$10/2$	5	0	4
$5/2$	2	1	5
$2/2$	1	0	6
$1/2$	0	1	7

So $174_{10} = 10101110_2$

Conversion steps:

1. Divide the number by 2.
2. Get the integer quotient for the next iteration.
3. Get the remainder for the binary digit.
4. Repeat the steps until the quotient is equal to 0.

*** Down -> top (নিচ থেকে উপরের দিকে সংখ্যা সাজানো)

Floating Number Decimal to Binary convert:

Top -> Down

13. Benefits of High level Language (HLL)

- Closer to Natural Languages
 - C++
 - FORTRAN scientific computation
 - COBOL business data processing
- Increased Programmer productivity
 - Small and concise code
 - Easy and Quick Programming
- Independent of Hardware Platform
 - Compiler converts HLL code to target code

14. Define **Instruction Set Architecture (ISA)** with example

- Different CPUs implement different sets of instructions. The set of instructions a particular CPU implements is an *Instruction Set Architecture (ISA)*.
 - Examples: Intel 80x86 (Pentium 4), IBM/Motorola PowerPC (Macintosh), MIPS, Intel IA64, ...

15. Explain **Reduced Instruction Set Computing**.

RISC (Reduced Instruction Set Computer) is a CPU design that uses a small number of simple and fast instructions to improve performance and efficiency.

RISC philosophy (John Cocke IBM, David A. Patterson, John L. Hennessy, 1980s) – **Reduced Instruction Set Computing**

- Keep the instruction set small and simple, makes it easier to build fast hardware.

Features of RISC:

- fixed instruction lengths
- load-store instruction sets
- limited addressing modes
- limited operations

16. Define **Microprocessor without Interlocked Pipeline Stages (MIPS)** Architecture. Why MIPS instead of Intel 80x86?

MIPS – semiconductor company that built one of the first commercial RISC architectures.

Instead of Intel 80x86:

- MIPS is simple, elegant. Don't want to get bogged down in gritty details.
- MIPS widely used in embedded apps, x86 little used in embedded, and more embedded computers than PCs
- NEC, Silicon Graphics, Nintendo, Sony etc.

17. **Assembly Variables : Registers**

a. Question: Drawback: Since registers are in hardware, there are a predetermined number of them

Solution: MIPS code must be very carefully put together to efficiently use registers.

Why Is This a Drawback?

1. Limited Number of Registers
2. Performance May Decrease: More memory access → More load/store instructions → Slower execution
3. Less Flexibility

b. Question: 32 registers in MIPS. Why 32?

Smaller is faster

In **MIPS architecture**, there are **32 general-purpose registers**. This design choice was made for a balance between **performance, hardware cost, and simplicity**.

Main reasons for choosing 32 registers:

1. Efficient Instruction Encoding

2. Good Balance Between Cost and Performance
3. Supports Compiler Optimization

18. MIPS Assembly Language

Arithmetic -> all

Data transfer -> load, store

Logical -> and, or

Conditional branch -> all

Unconditional jump -> jump, jump register

Category	Instruction	Example	Meaning
Arithmetic	add	add \$s1,\$s2,\$s3	$\$s1 = \$s2 + \$s3$
	subtract	sub \$s1,\$s2,\$s3	$\$s1 = \$s2 - \$s3$
	add immediate	addi \$s1,\$s2,20	$\$s1 = \$s2 + 20$
Data transfer	load word	lw \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$
	store word	sw \$s1,20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1$
	load half	lh \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$
	load half unsigned	lhu \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$
	store half	sh \$s1,20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1$
	load byte	lb \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$
	load byte unsigned	lbu \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$
	store byte	sb \$s1,20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1$
	load linked word	ll \$s1,20(\$s2)	$\$s1 = \text{Memory}[\$s2 + 20]$
	store condition. word	sc \$s1,20(\$s2)	$\text{Memory}[\$s2 + 20] = \$s1; \$s1 = 0$
Logical	load upper immedi.	lui \$s1,20	$\$s1 = 20 * 2^{16}$
	and	and \$s1,\$s2,\$s3	$\$s1 = \$s2 \& \$s3$
	or	or \$s1,\$s2,\$s3	$\$s1 = \$s2 \mid \$s3$
	nor	nor \$s1,\$s2,\$s3	$\$s1 = \sim (\$s2 \mid \$s3)$
	and immediate	andi \$s1,\$s2,20	$\$s1 = \$s2 \& 20$
	or immediate	ori \$s1,\$s2,20	$\$s1 = \$s2 \mid 20$
	shift left logical	sll \$s1,\$s2,10	$\$s1 = \$s2 \ll 10$
	shift right logical	srl \$s1,\$s2,10	$\$s1 = \$s2 \gg 10$
Conditional branch	branch on equal	beq \$s1,\$s2,25	if ($\$s1 == \$s2$) go to PC + 4 + 100
	branch on not equal	bne \$s1,\$s2,25	if ($\$s1 \neq \$s2$) go to PC + 4 + 100
	set on less than	slt \$s1,\$s2,\$s3	if ($\$s2 < \$s3$) $\$s1 = 1$; else $\$s1 = 0$
	set on less than unsigned	sltu \$s1,\$s2,\$s3	if ($\$s2 < \$s3$) $\$s1 = 1$; else $\$s1 = 0$
	set less than immediate	slti \$s1,\$s2,20	if ($\$s2 < 20$) $\$s1 = 1$; else $\$s1 = 0$
	set less than immediate unsigned	sltiu \$s1,\$s2,20	if ($\$s2 < 20$) $\$s1 = 1$; else $\$s1 = 0$
Unconditional jump	jump	j 2500	go to 10000
	jump register	jr \$ra	go to \$ra
	jump and link	jal 2500	$\$ra = \text{PC} + 4$; go to 10000

19. Format (R,I,J)

Name	Fields						Comments
Field size	6 bits	5 bits	5 bits	5 bits	5 bits	6 bits	All MIPS instructions are 32 bits long
R-format	op	rs	rt	rd	shamt	funct	Arithmetic instruction format
I-format	op	rs	rt	address/immediate			Transfer, branch, imm. format
J-format	op	target address					Jump instruction format

Instruction class	MIPS examples
Arithmetic	add, sub, addi
Data transfer	lw, sw, lb, lbu, lh, lhu, sb, lui
Logical	and, or, nor, andi, ori, sll, srl
Conditional branch	beq, bne, slt, slti, sltiu
Jump	j, jr, jal

20. MIPS register

Name	Reg. No.	Usage	Preserved on Call?
\$zero	0	const. value	n.a.
\$v0 - \$v1	2-3	values	No
\$a0 - \$a3	4-7	arguments	Yes
\$t0 - \$t7	8-15	temporaries	No
\$s0 - \$s7	16-23	saved	Yes
\$t8 - \$t9	24-25	temporaries	No
\$gp	28	Global pointer	Yes
\$sp	29	Stack pointer	Yes
\$fp	30	Frame Pointer	Yes
\$ra	31	Return address	Yes

21. Assembly Variable : Registers

- Registers are numbered from 0 to 31
- Each register can be referred to by number or name
- Number references:

\$0, \$1, \$2, ... \$30, \$31

*** By convention, each register also has a name to make it easier to code

- **For now:**

\$16 - \$23 → \$s0 - \$s7

(correspond to C variables)

\$8 - \$15 → \$t0 - \$t7

22. C , Java variables vs registers

Variables (C/Java)	Registers
1. Declared by programmer	1. Built into CPU hardware
2. Stored mainly in RAM	2. Stored inside CPU
3. Slower access	3. Very fast access
4. Can create many variables	4. Limited number (e.g., 32 in MIPS)
5. Example: int fahr, celsius; Char a,b,c,d,e;	5. Example: register -> \$ (dollar sign)

23. Assembly Instruction Short note:

- In assembly language, each statement (called an Instruction), executes exactly one of a short list of simple commands
- Unlike in C (and most other High Level Languages), each line of assembly code contains at most 1 instruction
- Instructions are related to operations (=, +, -, *, /) in C or Java

24. Addition and Subtraction of Integers:

- **Addition in Assembly**

- Example: add \$s0, \$s1,\$s2 (in MIPS)

Equivalent to: a = b + c (in C)

where MIPS registers \$s0, \$s1, \$s2 are associated with C variables a, b, c

- **Subtraction in Assembly**

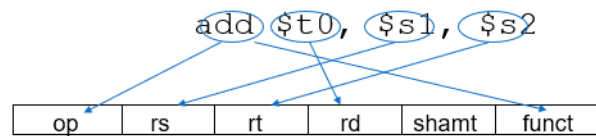
- Example: sub \$s3, \$s4,\$s5 (in MIPS)

Equivalent to: d = e - f (in C)

where MIPS registers \$s3, \$s4, \$s5 are associated with C variables d, e, f

25. Add Instruction:

- Instructions, like registers and words of data, are 32 bits long
- Arithmetic Instruction Format (R format):



op	6-bits	opcode that specifies the operation
rs	5-bits	register file address of the first source operand
rt	5-bits	register file address of the second source operand
rd	5-bits	register file address of the result's destination
shamt	5-bits	shift amount (for shift instructions)
funct	6-bits	function code augmenting the opcode

26. R-format example -> Machine code convert.

i. Assembly to Binary

ii. Binary to Assembly

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

Question: add \$t0, \$s1, \$s2

special	\$s1	\$s2	\$t0	0	add
0	17	18	8	0	32
000000	10001	10010	01000	00000	100000

$$00000010001100100100100000000100000_2 = 02324020_{16}$$

27. Addition and Subtraction of Integers

- How do the following C statement?

$$a = b + c + d - e;$$

- Break into multiple instructions

add \$t0, \$s1, \$s2

temp = b + c

add \$t0, \$t0, \$s3

temp = temp + d

sub \$s0, \$t0, \$s4

a = temp - e

- Notice: A single line of C may break up into several lines of MIPS.
- Notice: Everything after the hash mark on each line is ignored (comments)

28. Addition and Subtraction of Integers

- How do we do this?

$$f = (g + h) - (i + j);$$

- Use intermediate temporary register

add \$t0,\$s1,\$s2	<i># temp = g + h</i>
add \$t1,\$s3,\$s4	<i># temp = i + j</i>
sub \$s0,\$t0,\$t1	<i># f=(g+h)-(i+j)</i>

29. Register Zero

- One particular immediate, the number zero (0), appears very often in code.
- So we define register zero (\$0 or \$zero) to always have the value 0; eg

add \$s0, \$s1, \$zero (in MIPS)

f = g (in C)

where MIPS registers \$s0, \$s1 are associated with C variables f, g

- defined in hardware, so an instruction

add \$zero, \$zero, \$s0

30. Immediates

- Immediates are numerical constants.
- Add Immediate:

add \$s0, \$s1, 10 (in MIPS)

f = g + 10 (in C)

where MIPS registers \$s0, \$s1 are associated with C variables f, g

*** 27, 28, 29, 30 -> simple code segment দেওয়া থাকবে Assembly language convert করতে হবে .